
AMQP Protocol
Advanced Message Queuing Protocol
Panwit Tuwanut, IT KMITL

1. Message-Oriented Middleware

- ไม่มี Middleware
  - เป็นการสื่อสารแบบ Direct Communication
  - A ต้องรู้จัก B และทำการติดต่อ B โดยตรง และ A ต้องไปส่งให้ B โดยตรง
  - บางครั้ง B อาจจะไม่พร้อมรับ Message

- มี Message-Oriented Middleware
  - แทนที่จะให้ A คุยกับ B โดยตรง ก็ให้ส่ง Message ผ่านตัวกลาง
  - ตัวกลาง ทำหน้าที่ รับ–ส่ง Message ระหว่าง A กับ B โดยผู้ส่ง และผู้รับ ไม่จำเป็นต้องสื่อสารกันโดยตรง
  - ตัวกลาง จะช่วย รับ, เก็บ, จัดคิว, ส่งต่อ และจัดการ Message
  - ข้อดีคือ ผู้ส่งไม่จำเป็นต้องรู้ว่า ผู้รับอยู่ที่ไหนหรือพร้อมรับ Message หรือไม่
  - **ตัวอย่างในชีวิตจริง :** ไปรษณีย์

ข้อดีของ Message-Oriented Middleware

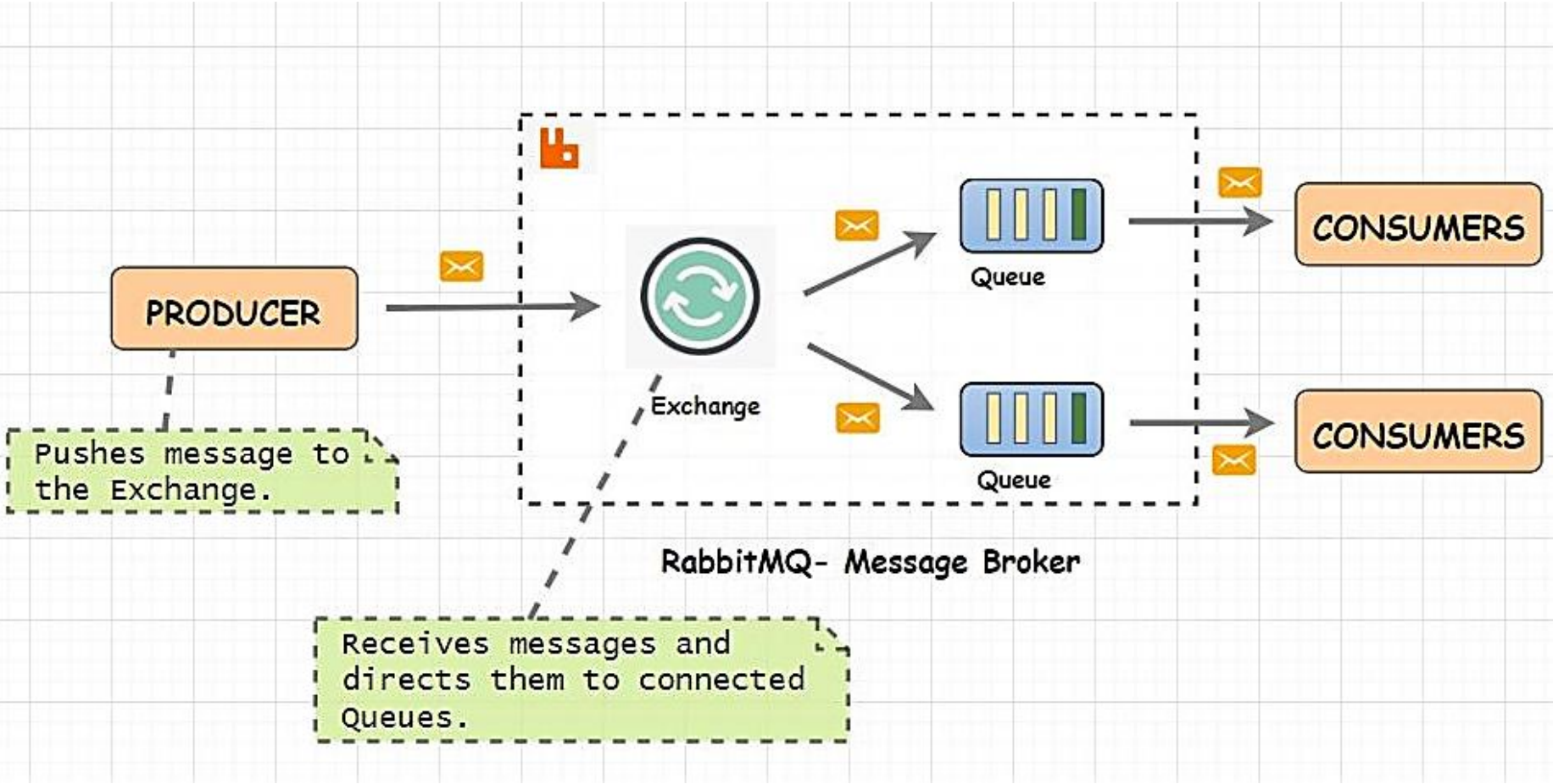
- ปัญหาของ Direct Communication
  - เมื่อ Server ไม่พร้อมทำงานจะส่งผลให้ Client ไม่สามารถส่งข้อมูลได้
  - Server และ Client ต้องทำงานพร้อมกัน
  - ระบบที่มี Client จำนวนมากจัดการยาก
  - การส่งข้อมูลจำนวนมากอาจทำให้ Server overload

- Message-Oriented Middleware ช่วยให้
  - ทำให้ ผู้ส่ง (Producer) และ ผู้รับ (Consumer) ทำงานแยกจากกันอย่างอิสระ
  - รองรับ **Asynchronous Communication**
  - Buffer ข้อมูลด้วย Queue
  - กระจายงานไปยังหลาย Consumer ได้

AMQP (Advanced Message Queuing Protocol)

- **AMQP เป็น**โปรโตคอลสำหรับการสื่อสารแบบ **Message-Oriented Middleware**
- ใช้แลกเปลี่ยน Message ระหว่าง Application
- รองรับ Asynchronous Communication
- มีระบบ Queue สำหรับพัก /จัดการ Message
- **ตัวอย่าง:** RabbitMQ

องค์ประกอบ AMQP



องค์ประกอบ

Component	หน้าที่
Producer	ผู้ส่ง Message
Consumer	ผู้รับ Message
Broker	ตัวกลางจัดการ Exchange และ Queue
Exchange	รับ Message และตัดสินใจว่าจะส่งไป Queue ใด โดยดูตามกฎ ที่ตั้งไว้
Queue	เก็บ Message รอ Consumer (เป็น Buffer) เป็น พื้นที่เก็บข้อความชั่วคราว รอให้ Consumer มารับไปประมวลผล
Routing Key	ค่าที่ใช้ตัดสินใจว่า เส้นทางของ Message จะไป Queue ใด
Binding	กฎการเชื่อมระหว่าง Exchange กับ Queue

อธิบายการทำงาน

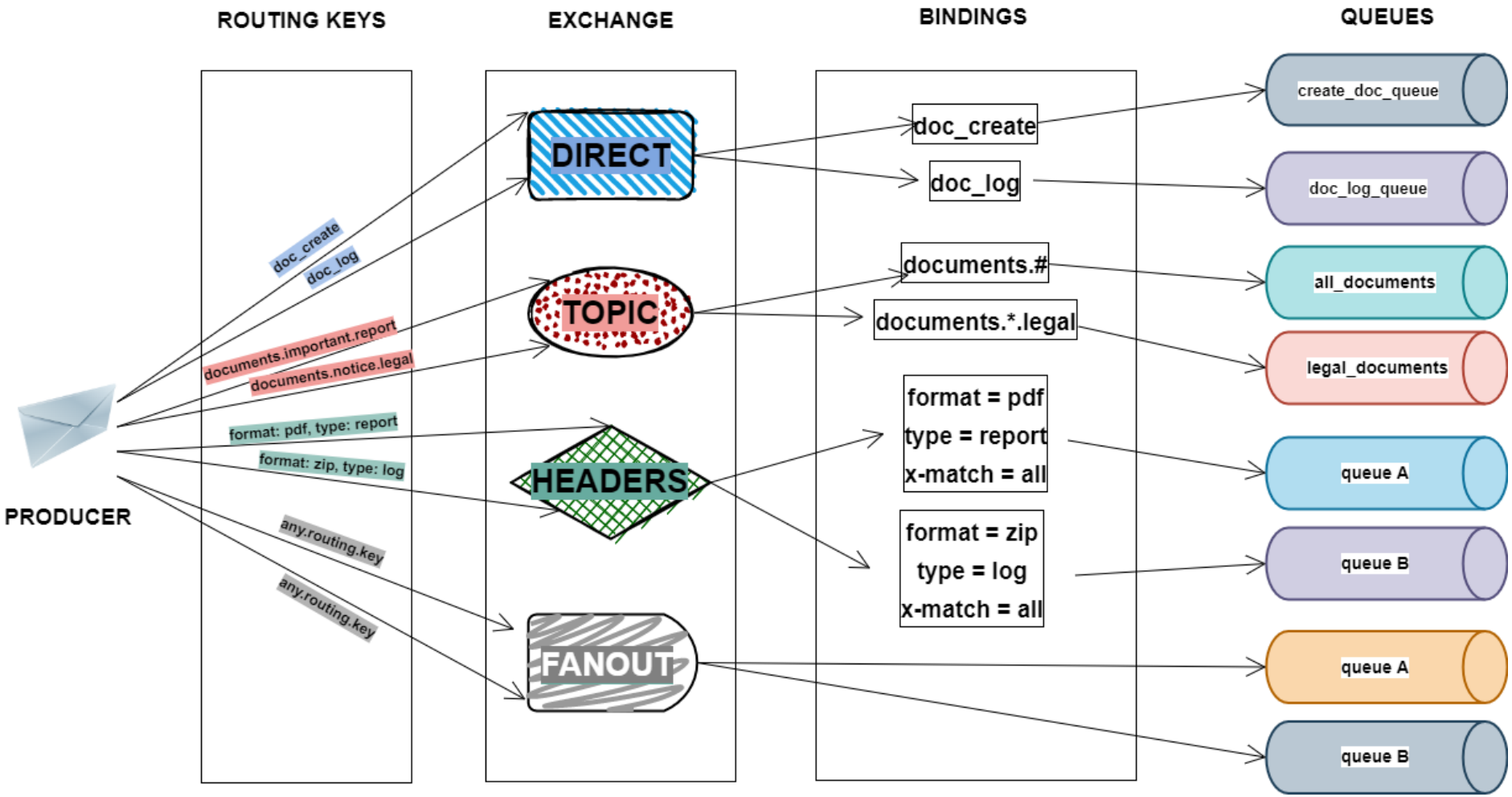
- Producer ทำการส่ง Message ไปยัง Exchange
  - ใน Message ที่ส่ง จะมีการระบุ Routing key เปรียบเสมือน Label ที่ติดมากับ Message
- Exchange เปรียบเสมือน ที่ทำการไปรษณีย์ ที่ทำการคัดแยก (ตัดสินใจ) ว่า จะส่ง Message ไปยัง Queue ไหน
  - โดยที่ Binding Key จะถูกกำหนดตอนสร้าง Binding ระหว่าง Queue กับ Exchange
  - ในการคัดแยก Message นั้น Exchange จะนำ Routing Key ของ Message ไปเปรียบเทียบกับ Binding Key ที่กำหนดไว้
  - หาก Routing Key ตรงกับ Binding Key ตามกฎของ Exchange นั้น ๆ Message จะถูกส่งไปยัง Queue ที่เกี่ยวข้อง
- Queue ทำหน้าที่เก็บ Message ไว้ชั่วคราวจนกว่า Consumer จะมารับ
- Consumer จะอ่านหรือรับ Message จาก Queue เพื่อนำไปประมวลผล

Note !! Direct : ต้องตรงกัน , Topic : ใช้ Wildcard , Fanout : ส่งทุก Queue, Headers : ใช้ Header แทน Routing Key

ประเภทของ Exchange

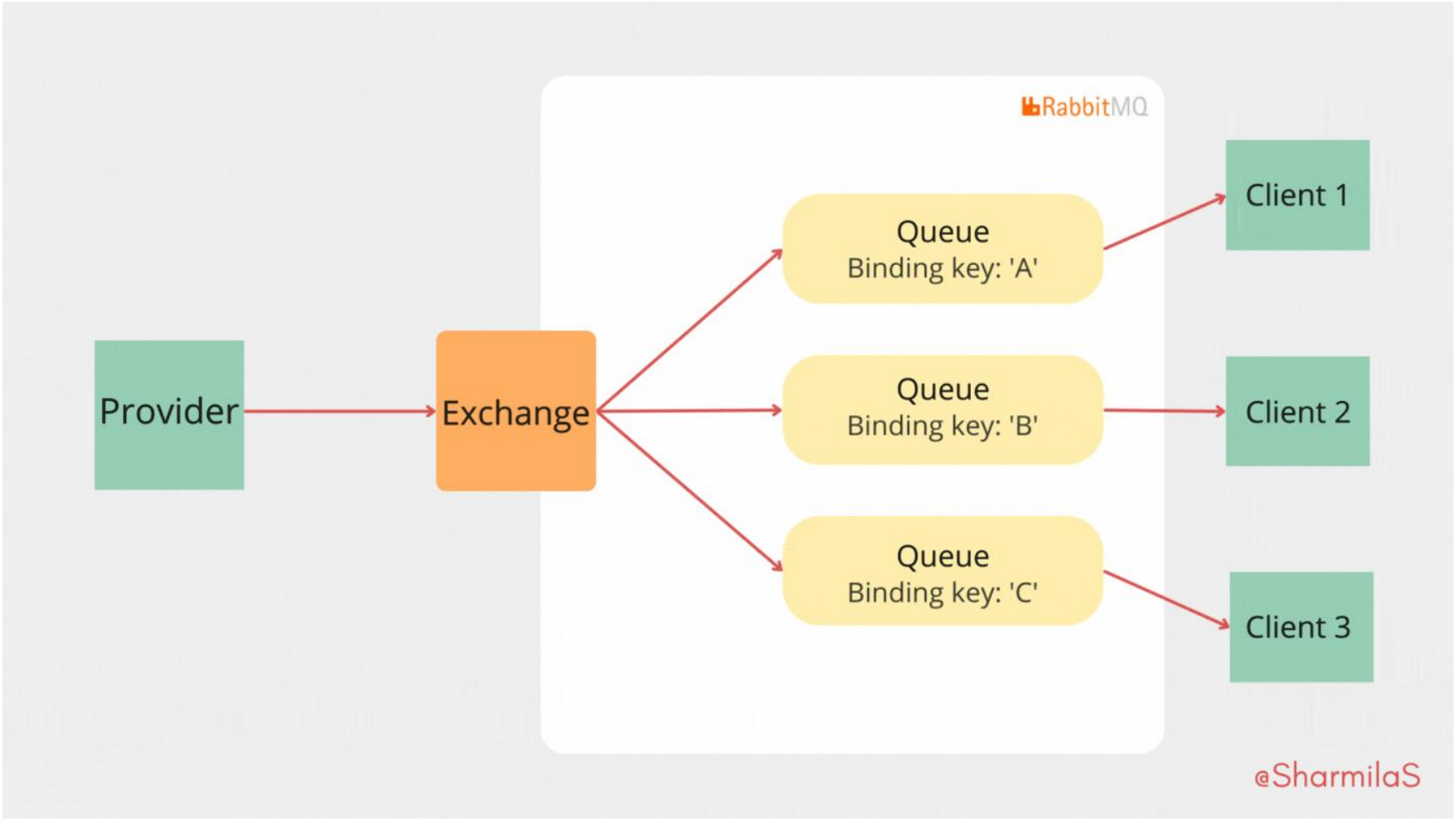
AMQP/RabbitMQ มี Exchange ที่สำคัญ 4 แบบ

- **Direct :** ส่งไป Queue ที่ routing key ตรงกับ Binding key
- **Fanout :** Broadcast ไปทุก Queue
- **Topic :** ใช้ pattern ของ routing key
- **Headers :** ใช้ Header ของ Message เป็นเงื่อนไขในการ Routing


ROUTING KEYS
EXCHANGE
BINDINGS
QUEUES
PRODUCER
doc_create
doc_log
documents.important.report
documents.notice.legal
format: pdf, type: report
format: zip, type: log
any.routing.key
any.routing.key
DIRECT
TOPIC
HEADERS
FANOUT
doc_create
doc_log
documents.#
documents.*.legal
format = pdf
type = report
x-match = all
format = zip
type = log
x-match = all
create_doc_queue
doc_log_queue
all_documents
legal_documents
queue A
queue B
queue A
queue B

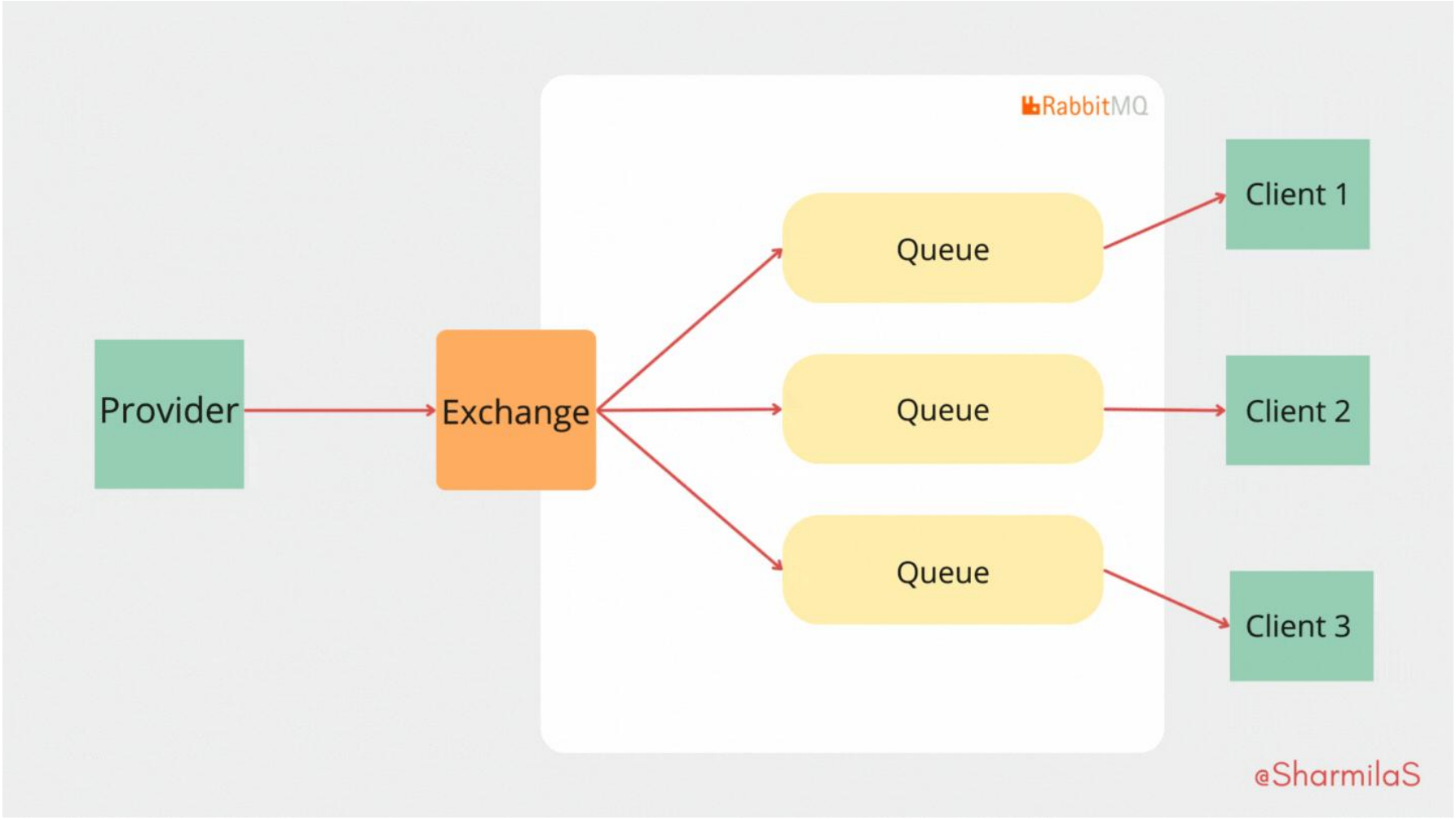
Type of Exchange

- Direct : นำ Routing Key มาเทียบกับ Binding key



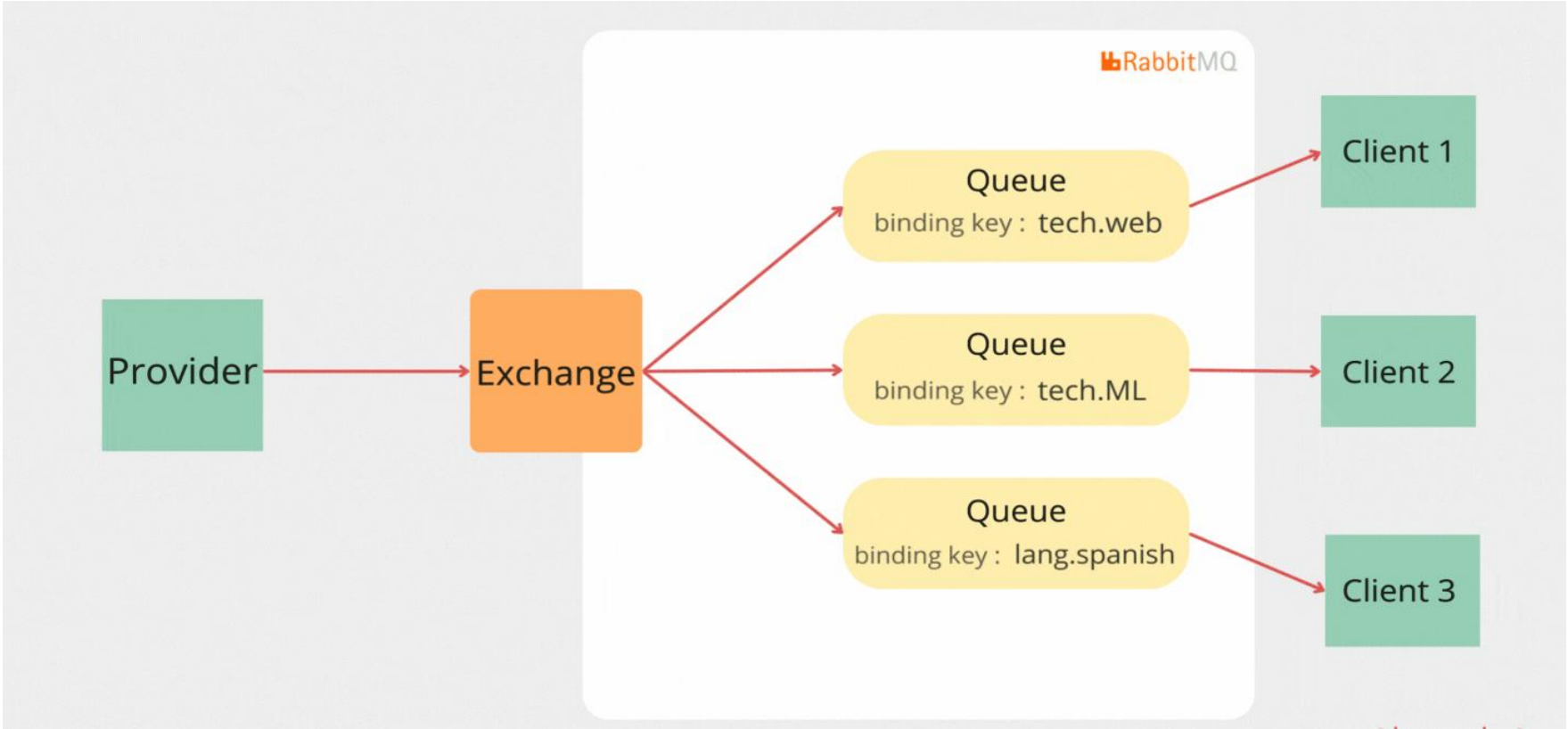
Type of Exchange

- Fanout : ทำการ broadcast message ไปยังทุก Queue



Type of Exchange

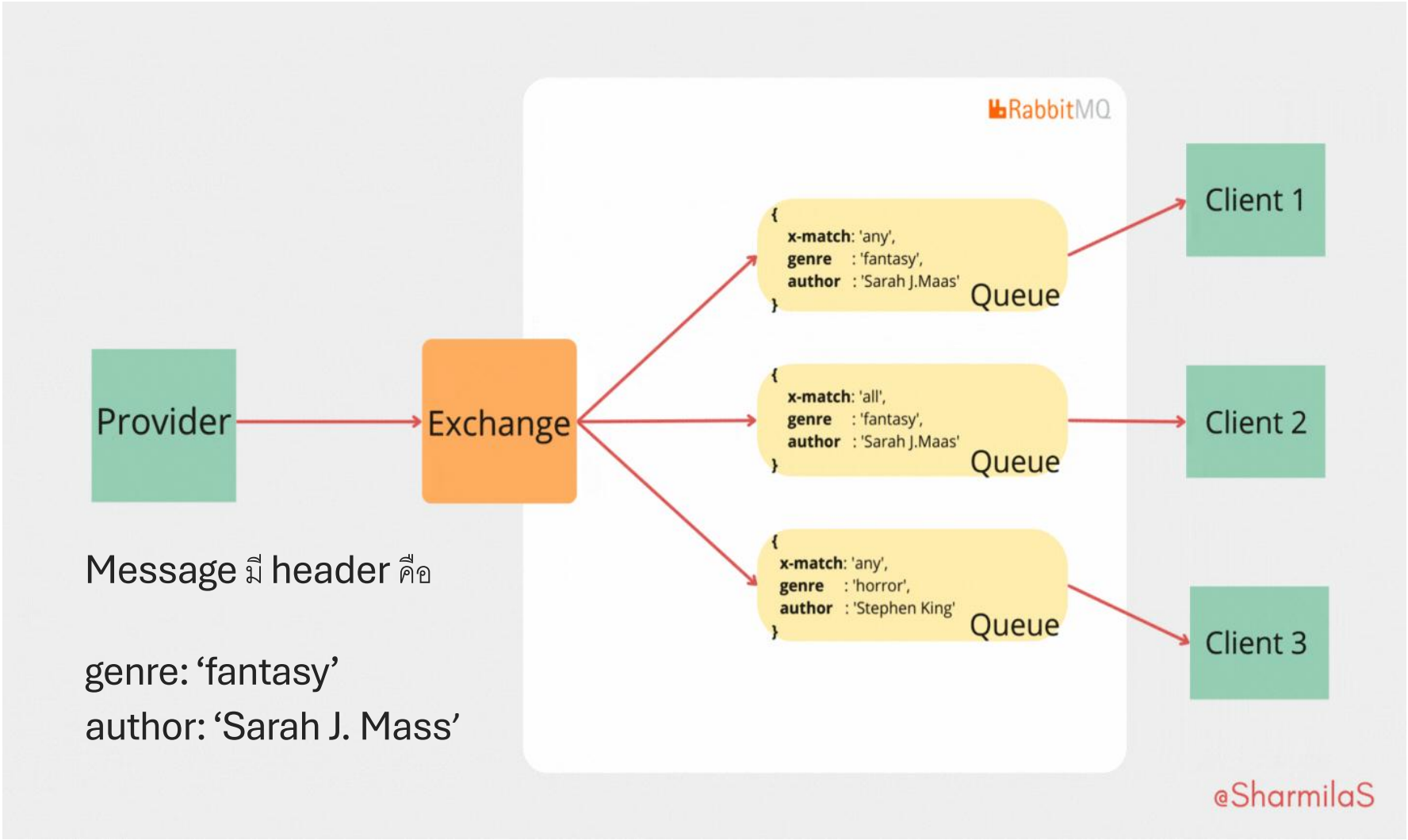
- Topic : routing key สามารถใช้ * หรือ # ในการ match ส่วนหนึ่งของ binding key
  - The asterisk (*) matches exactly one word.
  - The hash (#) symbol matches zero or more keys.
    - `sports.*` can represent `sports.tennis, sports.cricket`
    - `sports.#` can match `sports, sports.tennis, sports.cricket`



Type of Exchange

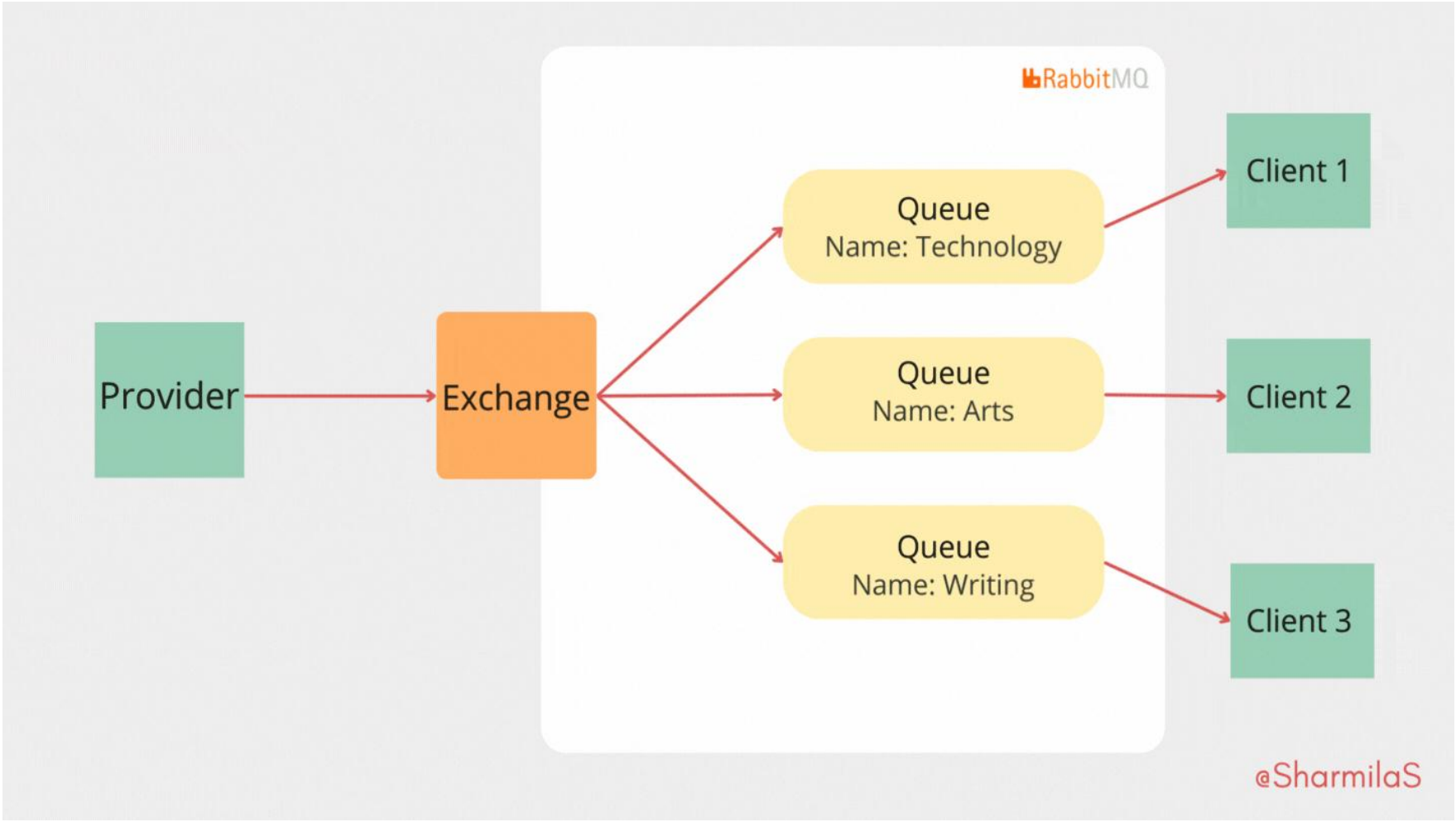
*x-match: '**all**' — all the header properties should match.*
*x-match: '**any**' — at least one header property should match.*

- Header : ใช้ header properties แทน routing keys และ binding keys.



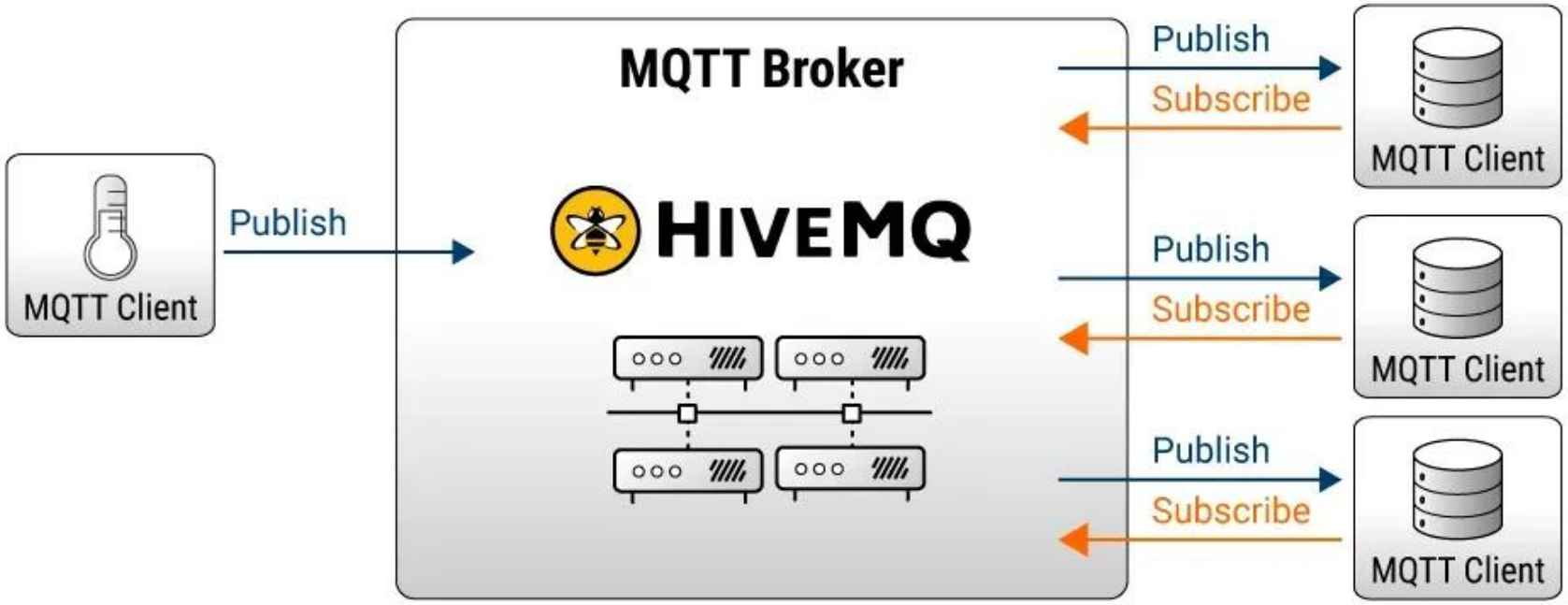
Type of Exchange

- Default : แทนที่จะใช้ binding key, จะใช้ **names** ของ queue ในการเปรียบเทียบกับ routing key เพื่อใช้ในการส่ง messages



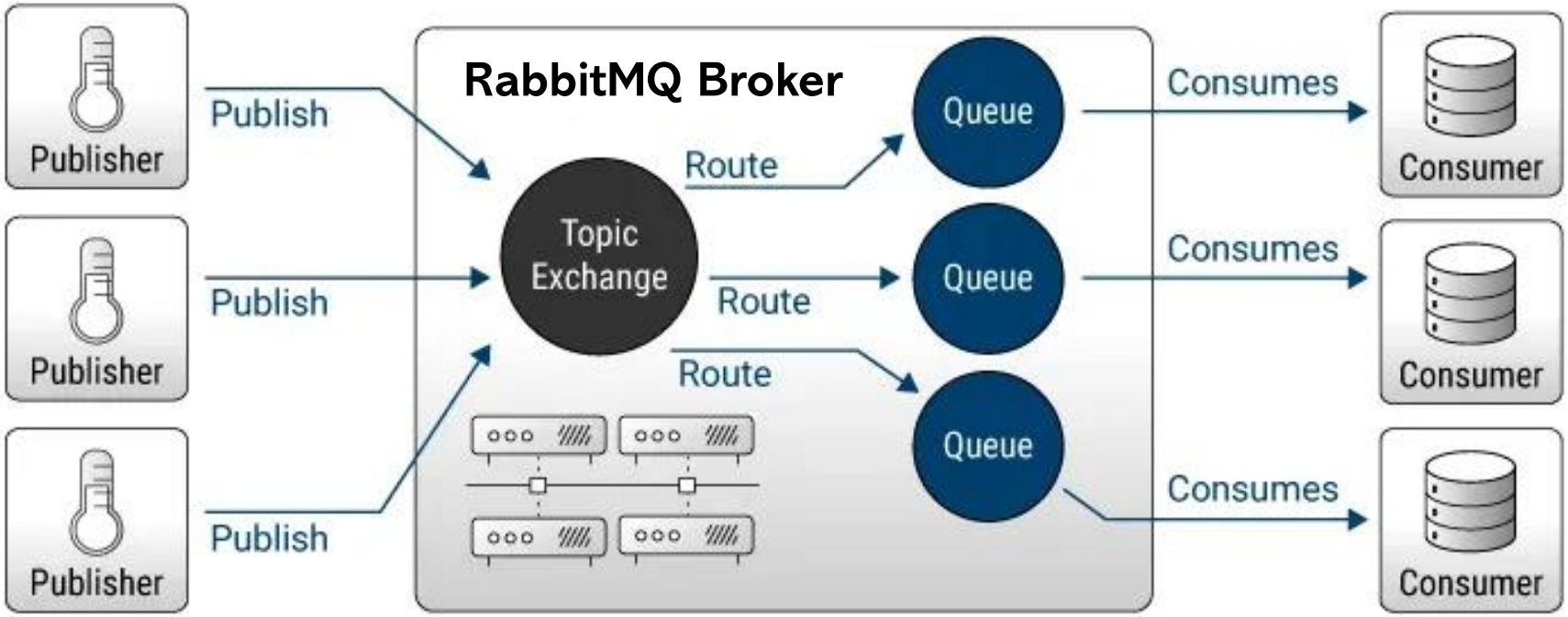
เปรียบเทียบ MQTT & AMQP

MQTT Topology



เปรียบเทียบ MQTT & AMQP

AMQP Topology



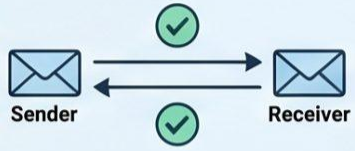
AMQP
(Advanced Message Queuing Protocol)

MQTT
(Message Queuing Telemetry Transport)

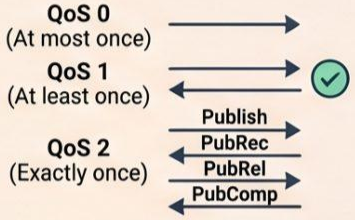
RELIABILITY PROTOCOLS COMPARISON: AMQP vs MQTT -
Focus on Message Acknowledgment, Transactions, and Guaranteed Delivery

Message Acknowledgment

Explicit Acknowledgment (ACK/NACK)
Supports individual, cumulative, and transactional acknowledgment.



Basic Acknowledgment
QoS Levels 0, 1, 2 define acknowledgment type.



Transactions

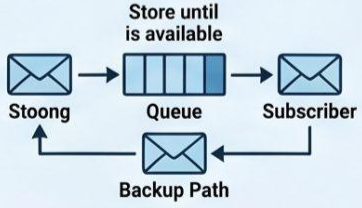
Built-in Support for Distributed Transactions (Two-Phase Commit)
Allows atomic operations on message publishing and consumption.



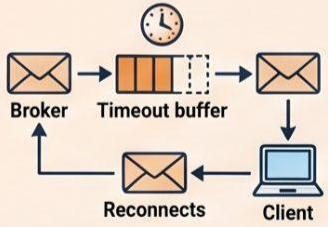
No Native Support for Complex Transactions
Message delivery is based on QoS, not atomic transactions.

Guaranteed Delivery

Strong Delivery Guarantees
Persistent Queues, Store-and-Forward, and Alternate Exchanges ensure message delivery even during network failures.

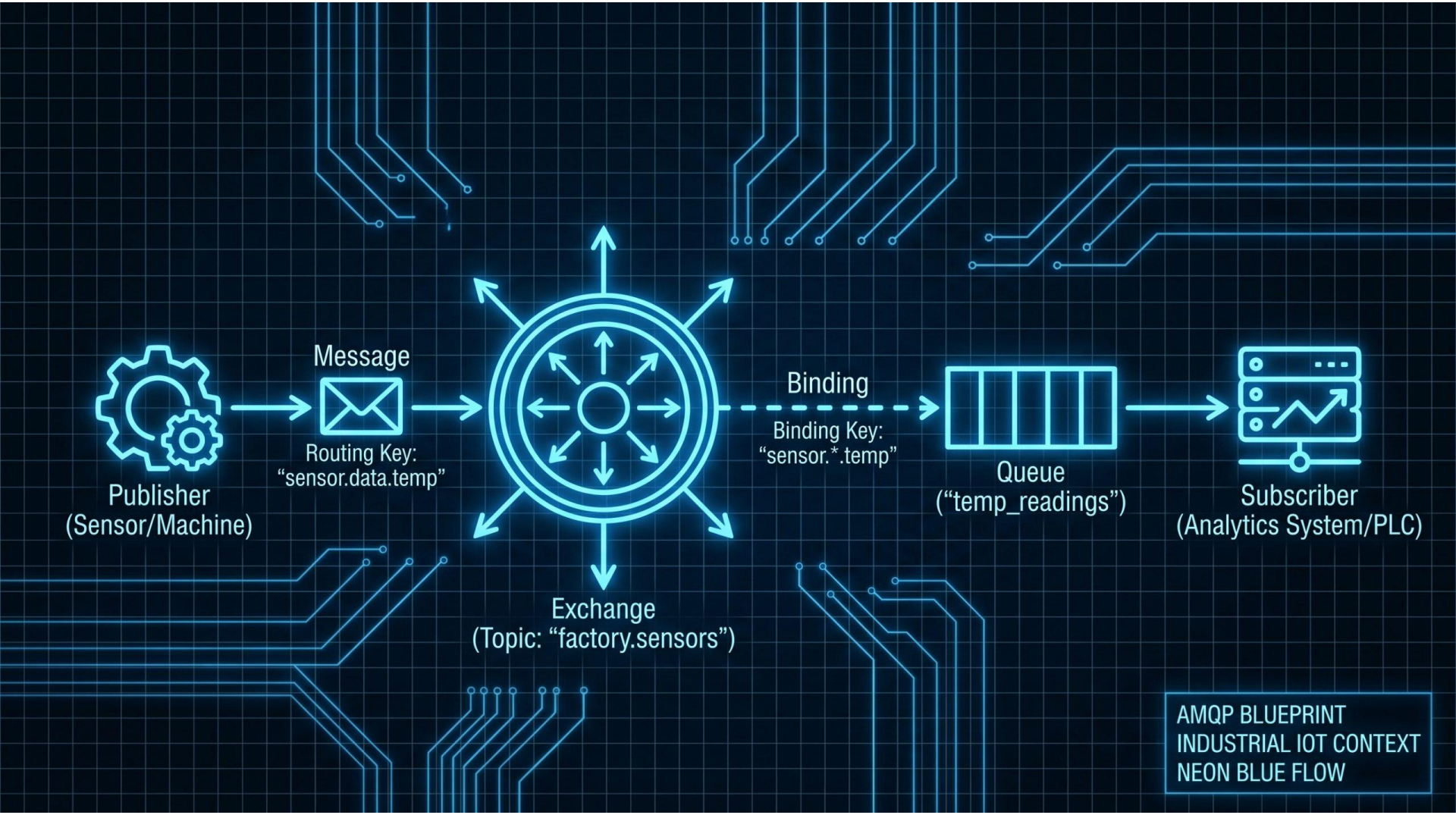


Guaranteed Delivery via QoS
Persistent Sessions store undelivered messages for QoS 1 and 2, but less robust than AMQP's queuing.



SUMMARY: AMQP offers comprehensive enterprise-grade reliability with advanced acknowledgment and transaction support, suitable for complex business processes. MQTT provides lightweight reliability optimized for constrained devices and networks via QoS levels, ideal for IoT and telemetry data.

IoT Application



AMQP in IoT

Reliable • Scalable • Decoupled • Real-time

IoT Devices (Producer)

- Temperature Sensor — 25.4 °C
- Humidity Sensor — 60 %
- Vibration Sensor — 2.3 mm/s
- Energy Meter — 12.5 kW
- IP Camera

Publish message (AMQP)

```
{
  "device_id": "M01",
  "type": "temperature",
  "value": 25.4,
} "timestamp": "2026-09-15T..."
```

IoT Gateway (Raspberry Pi) (AMQP Producer)

AMQP Broker (RabbitMQ)

RabbitMQ

Exchange

Routing by topic

- sensor.data (queue)
- alarm.queue (queue)
- video.queue (queue)

Routing Keys (topic)

- `sensor.temperature`
- `sensor.humidity`
- `sensor.vibration`
- `sensor.energy`
- `camera.video`

Applications (Consumers)

Data Storage (Consumer)

- Save to Database (MySQL / InfluxDB)
- Historical data

Alert & Notification (Consumer)

- Check threshold
- Send LINE / Email / SMS
- Create alarm event

Analytics / AI (Consumer)

- Real-time analytics
- Predictive maintenance
- Machine learning

Dashboard (Consumer)

- Real-time monitoring
- Visualization
- Web dashboard

Decoupling — Producer and Consumer are independent

Reliable Delivery — (ACK, Retry)

Flexible Routing — (exchange, routing key)

Scalable — multiple consumers

Asynchronous — handle burst message

Reliable & Secure — (TLS, Authentication)

AMQP – Powering Reliable IoT Systems

From Devices to Insights

การทดลอง

RabbitMQ



RabbitMQ

เข้าใช้งาน RabbitMQ Broker ได้ที่ https://amqp.iot.kmitl.co/

Username & Password ได้ส่งให้ทาง Email แล้ว

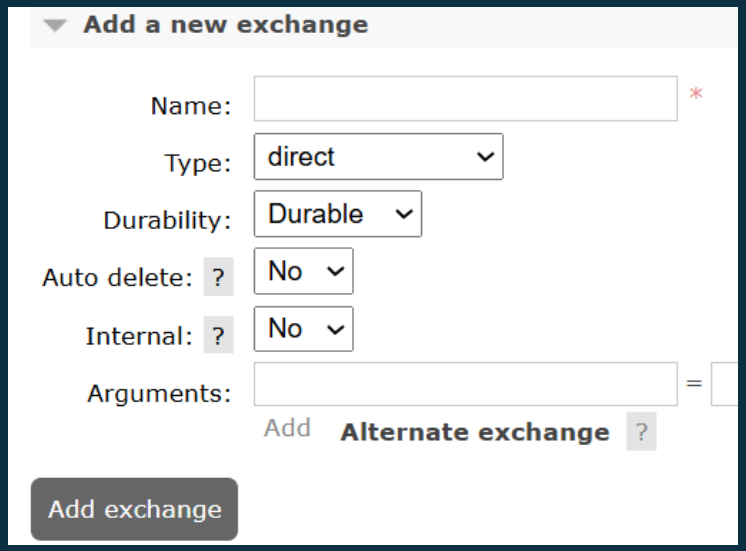


RabbitMQ

ทดลอง Exchanges

Direct : นำ Routing Key มาเทียบกับ Binding key

1. ไปที่ menu Exchanges
   - ทำการเพิ่ม Exchange โดยไปที่ Add a new exchange



- ตั้งชื่อ
- Type : direct
- Durability : Durable
- Auto delete : No
- Internal : No
- Argument :

- แล้วกด Add exchange

Durable : เมื่อปิด RabbitMQ **แล้วเปิดใหม่** Exchange/Queue นั้นจะยังอยู่

Transient : เมื่อ RabbitMQ restart จะทำให้ Queue นี้จะหายไป

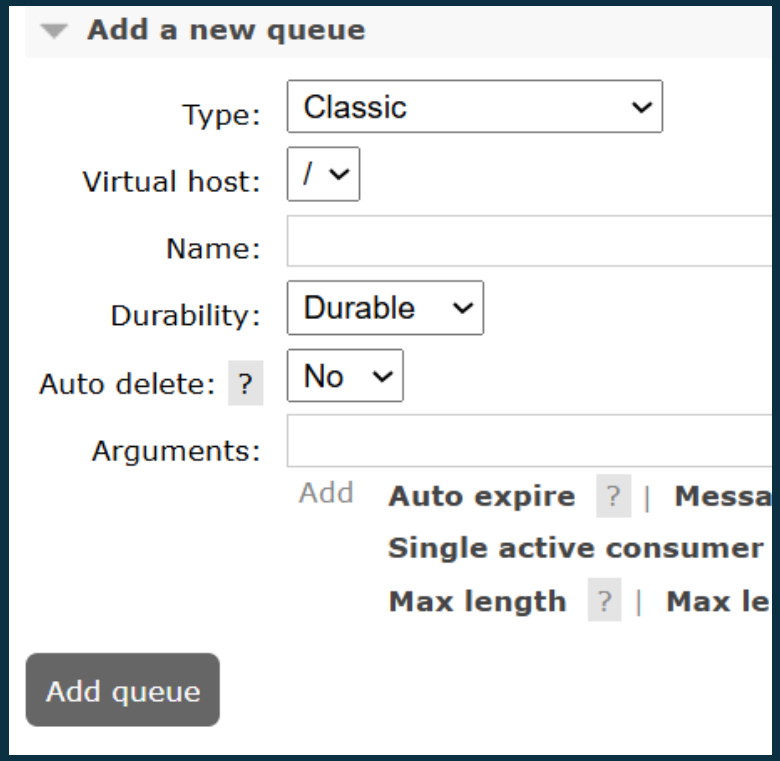
Note!! : Durable ≠ Message Persistent

RabbitMQ

ทดลอง Queues

2. ไปที่ menu **Queues and Streams**

- ทำการเพิ่ม Queues โดยไปที่ **Add a new queue**



- Type : Classic
- ตั้งชื่อ
- Durability : Durable
- Auto delete : No
- Argument :

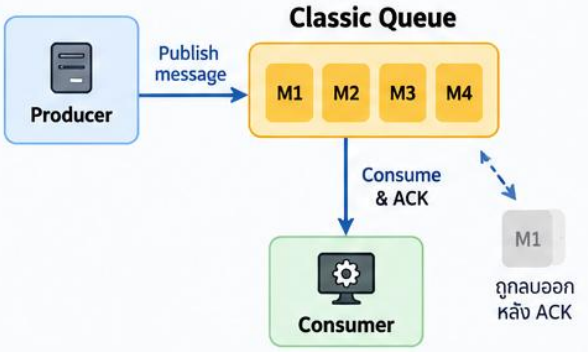
- แล้วกด Add queue

RabbitMQ | ความแตกต่างของ Queue ทั้ง 3 ประเภท

Same messaging, different purposes

Classic Queue

คิวพื้นฐาน – เหมาะกับงานทั่วไป



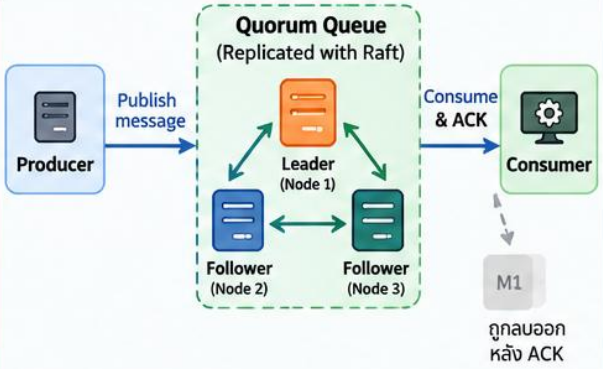
Replication	ไม่มี (เก็บที่ Node เดียว)
Durable	เลือกได้
Message หลัง ACK	ถูกลบออกจาก Queue
อ่านซ้ำ/Replay	ไม่ได้
หลาย Consumer อ่านข้อความเดียวกัน	ไม่ได้ (ต้องใช้หลาย Queue)
Throughput	ดี
nu Node ล่ม	ต่ำ

เหมาะสำหรับ

- งานทั่วไป (Task queue)
- ระบบที่ไม่ต้องการ High Availability สูงมาก
- ใช้ฟีเจอร์ของ Queue ได้ครบ

Quorum Queue

คิวที่เน้นความปลอดภัยและความพร้อมใช้งานสูง



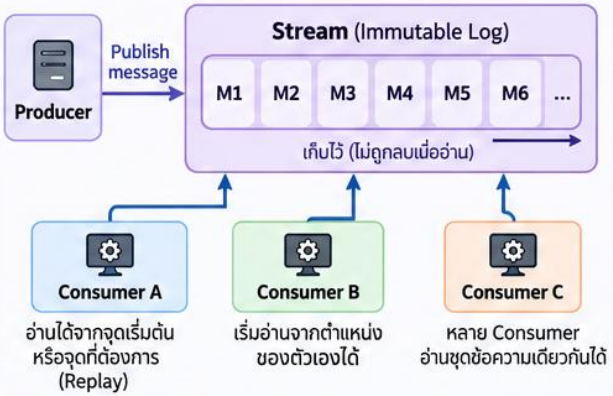
Replication	มี (หลาย Node ด้วย Raft)
Durable	บังคับ (เสมอ)
Message หลัง ACK	ถูกลบออกจาก Queue
อ่านซ้ำ/Replay	ไม่ได้
หลาย Consumer อ่านข้อความเดียวกัน	ไม่ได้ (ต้องใช้หลาย Queue)
Throughput	ดี
nu Node ล่ม	สูง (มีการเลือก Leader อัตโนมัติ)

เหมาะสำหรับ

- งานสำคัญ (เช่น ธุรกรรม)
- ระบบที่ห้ามข้อความหาย
- ต้องการ High Availability
- สภาพแวดล้อม Production

Stream

เก็บเหตุการณ์แบบลำดับเวลา – อ่านซ้ำได้



Replication	มี (หลาย Node)
Durable	บังคับ (เสมอ)
Message หลัง ACK	ยังคงเก็บไว้ (ไม่ถูกลบ)
อ่านซ้ำ/Replay	ได้ (อ่านย้อนหลังได้)
หลาย Consumer อ่านข้อความเดียวกัน	ได้ (Consumer แต่ละตัวมีตำแหน่งอ่านของตัวเอง)
Throughput	ดีมาก (ออกแบบมาสำหรับปริมาณมาก)
nu Node ล่ม	สูง

เหมาะสำหรับ

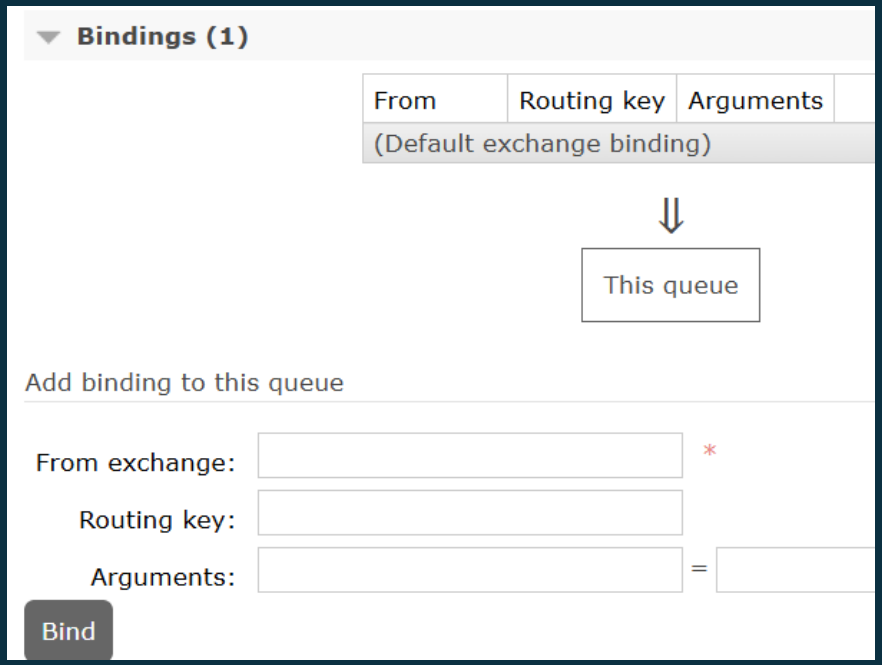
- การเก็บเหตุการณ์ (Event streaming)
- งานที่ต้องการอ่านย้อนหลัง/วิเคราะห์ข้อมูลในอดีต
- มี Consumer หลายตัวอ่านข้อมูลชุดเดียวกัน
- ปริมาณข้อความจำนวนมาก (High throughput)

"Classic ใช้งานทั่วไป | Quorum เพื่อความปลอดภัย | Stream เพื่อการเก็บเหตุการณ์และการอ่านย้อนหลัง"

RabbitMQ

ทดลอง Binding

3. ไปที่ menu Queues and Streams
   - เลือก Queue ที่ได้สร้างไว้ ในข้อ 2
   - เลื่อนลงมาหา เมนู Bindings



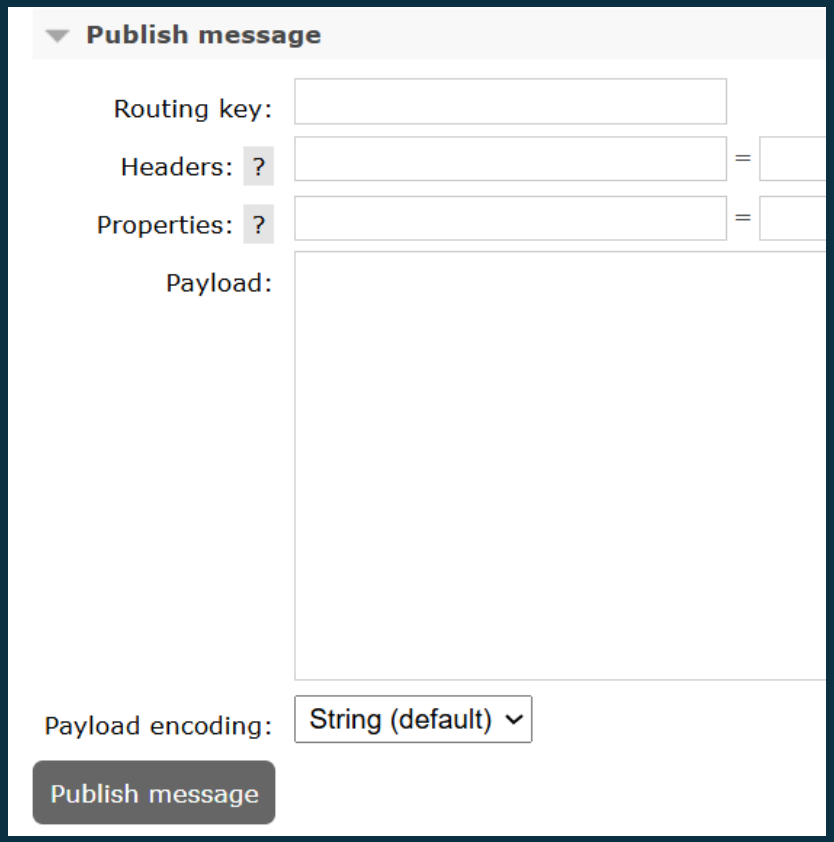
- ใส่ค่า
- From exchanges : ใส่ค่าตามข้อ 1
- Routing key :
  เช่น red

แล้วกดปุ่ม Bind

RabbitMQ

ทดลอง Publish

4. ไปที่ menu Exchanges
   - เลือก Exchange ที่ได้สร้างไว้ ในข้อ 1
   - เลื่อนลงมาหา เมนู **Publish message**



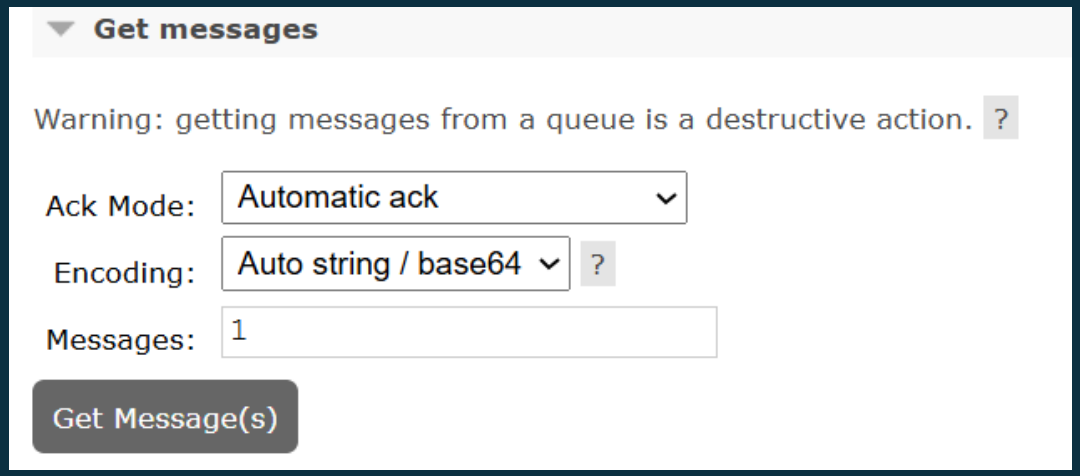
- ใส่ค่า
- Routing key : ใส่ค่าตามข้อ 3
- Payload :

แล้วกดปุ่ม Publish message

RabbitMQ

ทดลอง Get Message

5. ไปที่ menu Queues and Streams
   - เลือก Queue ที่ได้สร้างไว้ ในข้อ 2
   - เลื่อนลงมาหา เมนู **Get message**



- Ack Mode : Automatic ack
- Encoding : Auto string

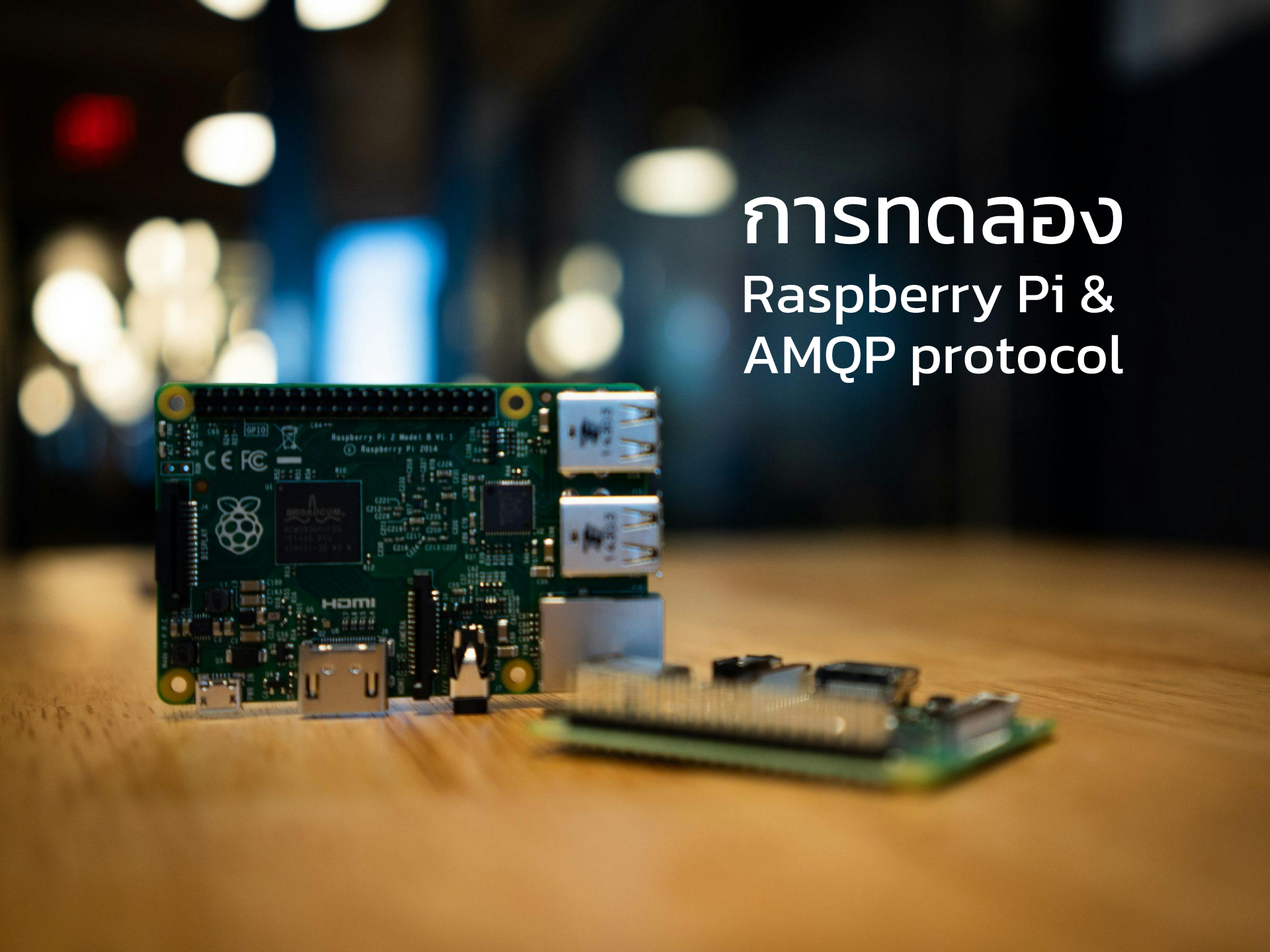
แล้วกดปุ่ม Get message(s)

ทดลองเล่นเอง ดังต่อไปนี้

- ในหน้า Publish message : ลองเปลี่ยน Routing key เป็นค่าอื่นๆ เช่น `yellow, red.*` แล้วลอง Publish แล้วดูว่า Get message ได้ไหม ?

- ในหน้า get message : ลองเปลี่ยน Ack mode เป็น NACK message แล้วดูว่า ค่าใน queue เป็นอย่างไร ?

- ลองเล่น Exchange ในรูปแบบอื่น ๆ เช่น Topic, Fanout , header


การทดลอง
Raspberry Pi &
AMQP protocol

การทดลอง. ติดตั้ง Library pika

```
$ sudo apt update
$ sudo apt install python3-pika
```

Pika is a pure-Python implementation of the AMQP 0-9-1 protocol that tries to stay fairly independent of the underlying network support library. Pika was developed primarily for use with RabbitMQ, but should also work with other AMQP 0-9-1 brokers.

https://pypi.org/project/python3-pika/

Method พื้นฐาน

Method	ฝั่ง	หน้าที่
`BlockingConnection()`	ทั้งคู่	สร้าง Connection ระหว่าง Python กับ RabbitMQ
`channel()`	ทั้งคู่	หลังจาก Connect แล้ว เราสร้าง Channel
`exchange_declare()`	ทั้งคู่	สร้าง Exchange
`queue_declare()`	ทั้งคู่	สร้าง Queue
`basic_publish()`	`Producer`	ส่ง Message
`basic_consume()`	`Consumer`	รับ Message
`basic_ack()`	`Consumer`	ยืนยันสำเร็จ
`basic_nack()`	`Consumer`	ปฏิเสธ/Retry

Code ตัวอย่าง producer.py , consumer.py

แก้ Code ในส่วนต่อไปนี้

```
RABBITMQ_USER = "รหัส นศ"
RABBITMQ_PASS = "ค่าที่กำหนดให้ของแต่ละคน"

EXCHANGE_NAME = "amq.direct"
EXCHANGE_TYPE = "direct"

ROUTING_KEY = "red"

virtual_host="/รหัส นศ",
```

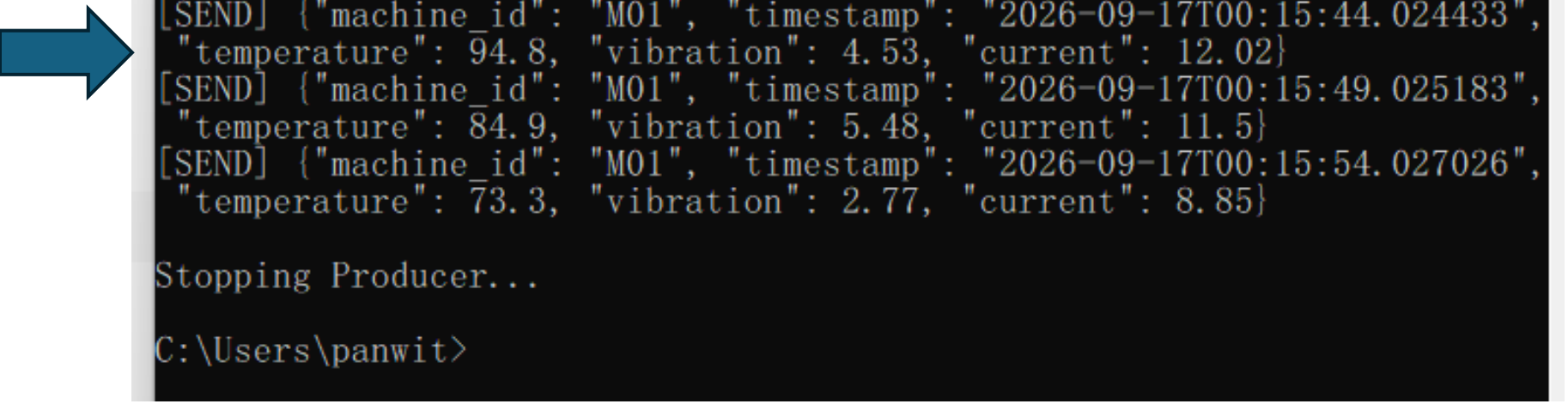
แก้เป็นชื่อ Exchange ของตัวเอง

แก้เป็นชื่อ Exchange ของตัวเอง

แก้เป็นชื่อ ROUTING_KEY ของตัวเอง ให้ตรงกับ Binding key ของแต่ละ Queue

การทดลองที่ 1. run ไฟล์ producer.py

ใช้ RabbitMQ Broker : Get Message (ไปที่เมนู Queue)



การทดลองที่ 1. run ไฟล์ producer.py

ใช้ RabbitMQ Broker : Get Message (ไปที่เมนู Queue)

Get messages

Warning: getting messages from a queue is a destructive action. ?

Ack Mode: Automatic ack

Encoding: Auto string / base64 ?

Messages: 1

Get Message(s)

Message 1

The server reported 2 messages remaining.

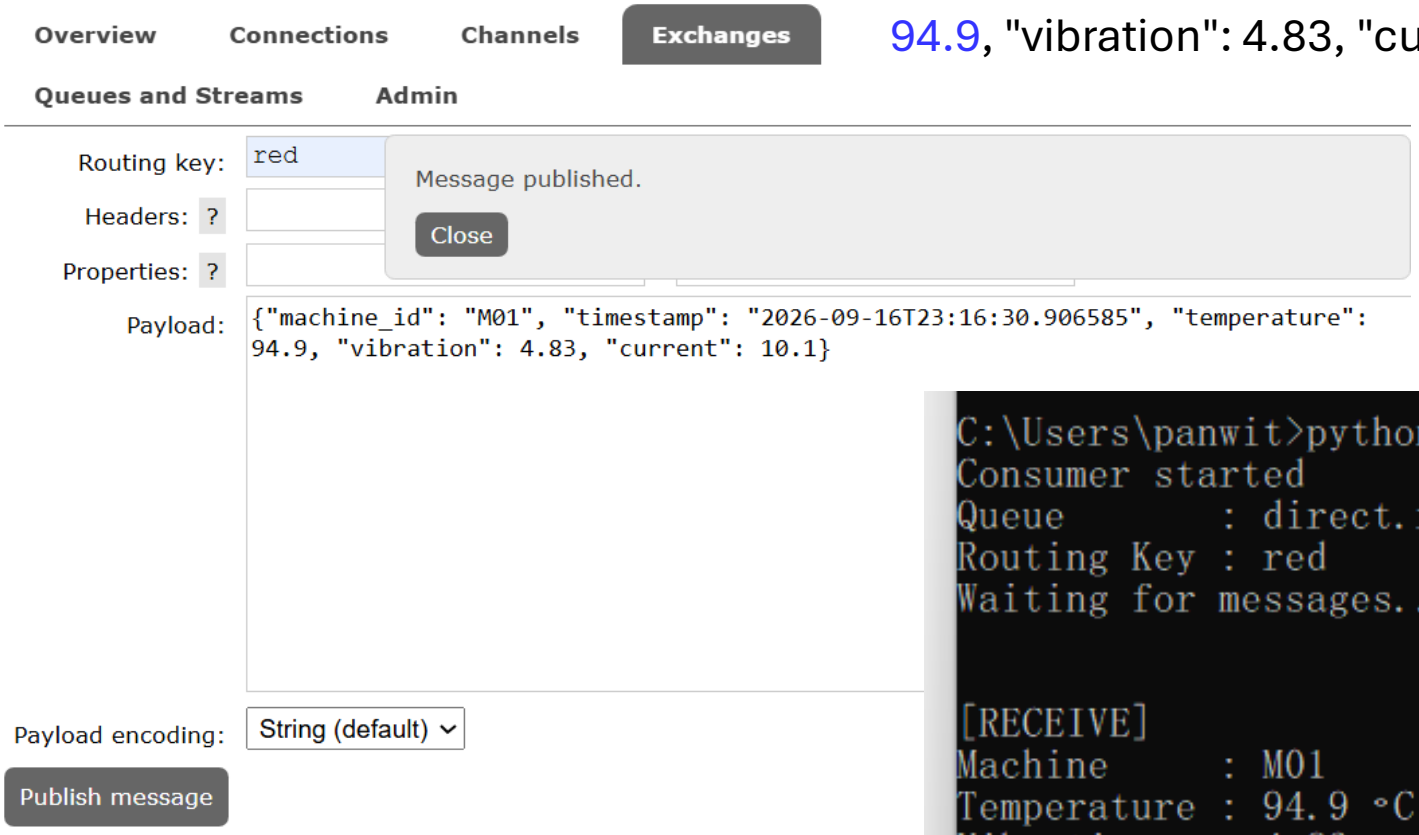
Exchange	amq.direct
Routing Key	red
Redelivered	○
Properties	delivery_mode: 2 content_type: application/json
Payload 122 bytes Encoding: string	{"machine_id": "M01", "timestamp": "2026-09-17T00:15:44.024433", "temperature": 94.8, "vibration": 4.53, "current": 12.02}

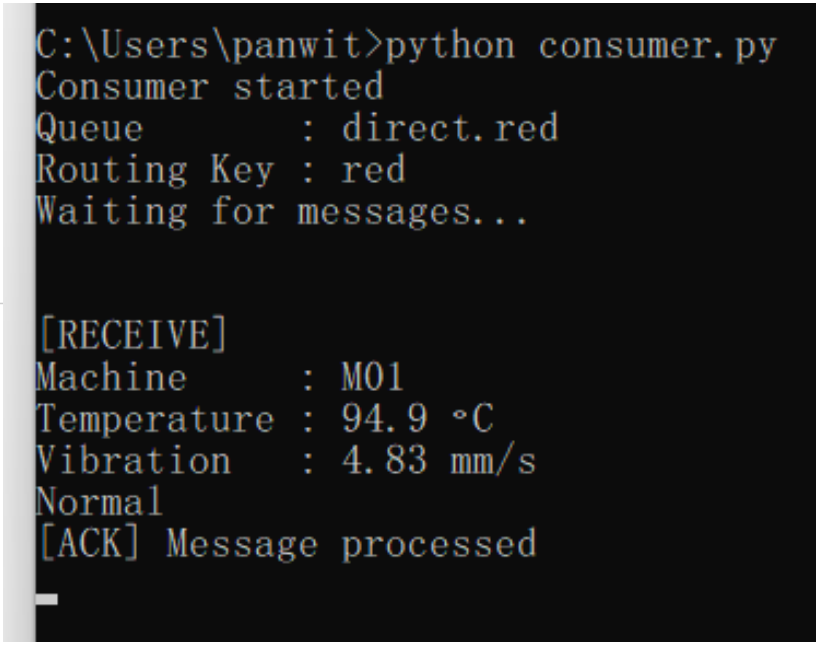


การทดลองที่ 2. run ไฟล์ cousumer.py

ใช้ RabbitMQ Broker : Publish JSON (ไปที่เมนู Exchange)

{"machine_id": "M01", "timestamp": "2026-09-16T23:16:30.906585", "temperature": 94.9, "vibration": 4.83, "current": 10.1}





การทดลองที่ 2. run ไฟล์ cousumer.py

ใช้ RabbitMQ Broker : Publish JSON (ไปที่เมนู Exchange)

{"machine_id": "M01", "timestamp": "2026-09-16T23:16:30.906585", "temperature": 104.9, "vibration": 4.83, "current": 10.1}

Overview | Connections | Channels | **Exchanges** | Queues and Streams | Admin

Routing key: red

Headers: ? = String

Properties: ? =

Payload:

```
{"machine_id": "M01", "timestamp": "2026-09-16T23:16:30.906585", "temperature":
104.9, "vibration": 4.83, "current": 10.1}
```

Payload encoding: String (default)

Publish message

```
[RECEIVE]
Machine     : M01
Temperature : 104.9 °C
Vibration   : 4.83 mm/s
!!! ALARM !!!
Temperature too high
[ACK] Message processed
```

แบบฝึกหัด

- สร้าง Producer 2 ตัว ส่งค่าอุณหภูมิ (random) วันเวลา

  ตัวที่ 1 ส่งทุก ๆ 2 วินาที

  ตัวที่ 2 ส่งทุก ๆ 3 วินาที

และมี Comsumer 2 ตัว

ตัวที่ 1 แสดงผลทุกค่าจากทั้ง 2 ตัว

ตัวที่ 2 แสดงผลเฉพาะค่าที่เกิน 50 องศา


END